Unity PSG Player - スクリプトリファレンス

- [class]
  - PSG Player
  - MML Spltter

[class] PSG Player

[PSG Player] 変数・Public関数一覧

- [変数](#[PSG Player] 変数)
  - mmlDecoder
  - audioSource
  - sampleRate
  - audioClipSizeMilliSec
  - a4Freq
  - tickPerNote
  - programChange
  - seqListIndex
  - seqList
  - mmlString
  - asyncRenderIsDone
  - asyncRenderProgress
  - renderedDatas
- [Public関数](#[PSG Player] public関数)
  - Play()
  - Play(string _mmlString)
  - DecodeMML()
  - PlayDecoded()
  - PlaySequence()
  - Stop()
  - IsPlaying()
  - Mute(bool isOn)
  - ExportSeqJson(bool _prettyPrint)
  - DecodeAndExportSeqJson(bool _prettyPrint)
  - GetSeqJson()
  - ImportSeqJson(string _jsonString)
  - SetSeqJson(SeqJson _seqJson)
  - RenderSequenceTodClipData()
  - ExportRenderedAudioClip(bool isAsyncRendered)
  - RenderSeqToClipDataAsync(int interruptSample)
  - CalcSeqSample()
  - PlayRenderedClipData(bool isLoop)

[PSG Player] 変数

---

mmlDecoder

```
[SerializeField] private MMLDecoder mmlDecoder;
```

MMLをシーケンスデータに変換するMML Decoderコンポーネントを登録します。

---

audioSource

```
[SerializeField] private AudioSource audioSource;
```

生成したAudioClipを渡すAudioSourceを登録します。

---

sampleRate

```
public int sampleRate = 32000;
```

AudioClipのサンプルレートを設定します。
デフォルトは32000Hz（32kHz）です。

---

audioClipSizeMilliSec

```
public int audioClipSizeMilliSec = 1000;
```

AudioClipの長さを設定します。
デフォルトは1000msec（1秒）です。

---

a4Freq

```
public float a4Freq = 440f;
```

音階の基準となるオクターブ4のラ（o4a）の音の周波数を設定します。
デフォルトは440Hzです。
この変数はMMLのコマンドで変更することができます。

---

tickPerNote

```
public int tickPerNote = 960;
```

1拍（4分音符）の分解能を設定します。
音長はこの分解能に基づいたティック数に変換され、実際の音の長さ（秒）はテンポとこの分解能から計算されます。

(例：8分音符は480ティックとなり、テンポ120の場合音の長さは
60[sec] / 120[notePerMin] * 480[tick] / 960[tickPerNote] = 0.25[sec]
で0.25秒となります。)

---

programChange

```
public int programChange;
```

生成する音色の番号です。

番号	波形
0	パルス波（12.5%）
1	パルス波（25%）
2	矩形波（50%）
3	パルス波（75%）
4	三角波
5	ノイズ
6	短周期ノイズ

この変数はMMLのコマンドで変更することができます。

---

seqListIndex

```
[SerializeField] private int seqListIndex = 0;
```

シーケンスデータの処理中の位置です。
主にデバッグ用途で表示します。

---

seqList

```
[SerializeField] private List<SeqEvent> seqList = new();
```

シーケンスデータのList配列です。
主にデバッグ用途で表示します。

---

mmlString

```
[Multiline] public string mmlString = "";
```

演奏するMML文字列です。
この変数をMML Decoderに渡してシーケンスデータに変換します。

---

asyncRenderIsDone

```
public bool asyncRenderIsDone { get; private set; } = false;
```

`v0.9.6beta`

非同期レンダリングが完了すると`True`になります。
RenderSeqToClipDataAsync()を呼び出した後に使用します。

---

asyncRenderProgress

```
public float asyncRenderProgress { get; private set; } = 0f;
```

`v0.9.6beta`

非同期レンダリングの進捗率で0から1の範囲で増加します。
RenderSeqToClipDataAsync()を呼び出した後に使用します。

---

renderedDatas

```
public float[] renderedDatas { get; private set; }
```

`v0.9.6beta`

RenderSequenceTodClipData()またはRenderSeqToClipDataAsync()で生成されたクリップデータが入ります。

[PSG Player] Public関数

Play()

```
public void Play();
```

- パラメーター：なし

mmlStringのMML文字列をシーケンスデータに変換して、再生を開始します。

Play(string _mmlString)

```
public void Play(string _mmlString);
```

- パラメーター：**_mmlString**　MML文字列

パラメーターの引数をmmlString変数に渡して、シーケンスデータに変換して再生を開始します。

DecodeMML()

```
public bool DecodeMML();
```

- パラメーター：なし
- 戻り値：デコード成功で`True`

mmlStringのMML文字列をMML Decoderに渡してシーケンスデータに変換します。

PlayDecoded()

```
public void PlayDecoded();
```

- パラメーター：なし

デコード済みのシーケンスデータを再生します。
変換処理を行わないので、CPU負荷の軽減が期待できます。

PlaySequence()

```
public void PlaySequence();
```

- パラメーター：なし

デコード済みのシーケンスデータを再生します。
PlayDecoded()と同じです。

Stop()

```
public void Stop();
```

- パラメーター：なし

再生中の音を停止します。

IsPlaying()

```
public bool IsPlaying();
```

- パラメーター：なし
- 戻り値：再生中なら`True`

AudioSourceの再生状況を返します。

Mute(bool isOn)

```
public void Mute(bool isOn);
```

- パラメーター：**isOn**　`True`ならミュートオン

~~ミュートをオンにすると、AudioSourceをミュートにした上で、生成されるサンプルを0（無音）にします。~~
~~`False`で解除すると、AudioSourceは即時にミュート解除されますが、サンプルは次のノートイベントが発生するまで無音を継続します。~~
~~ただし、バッファ済みのサンプルより早くミュート解除した場合は、すでに生成されたサンプルが発音されます。~~

`v0.9.7beta`

`True`でAudioSourceを即時ミュートオン、`False`で即時ミュートオフになります。
つまり、普通のミュートと同じ挙動になりました。

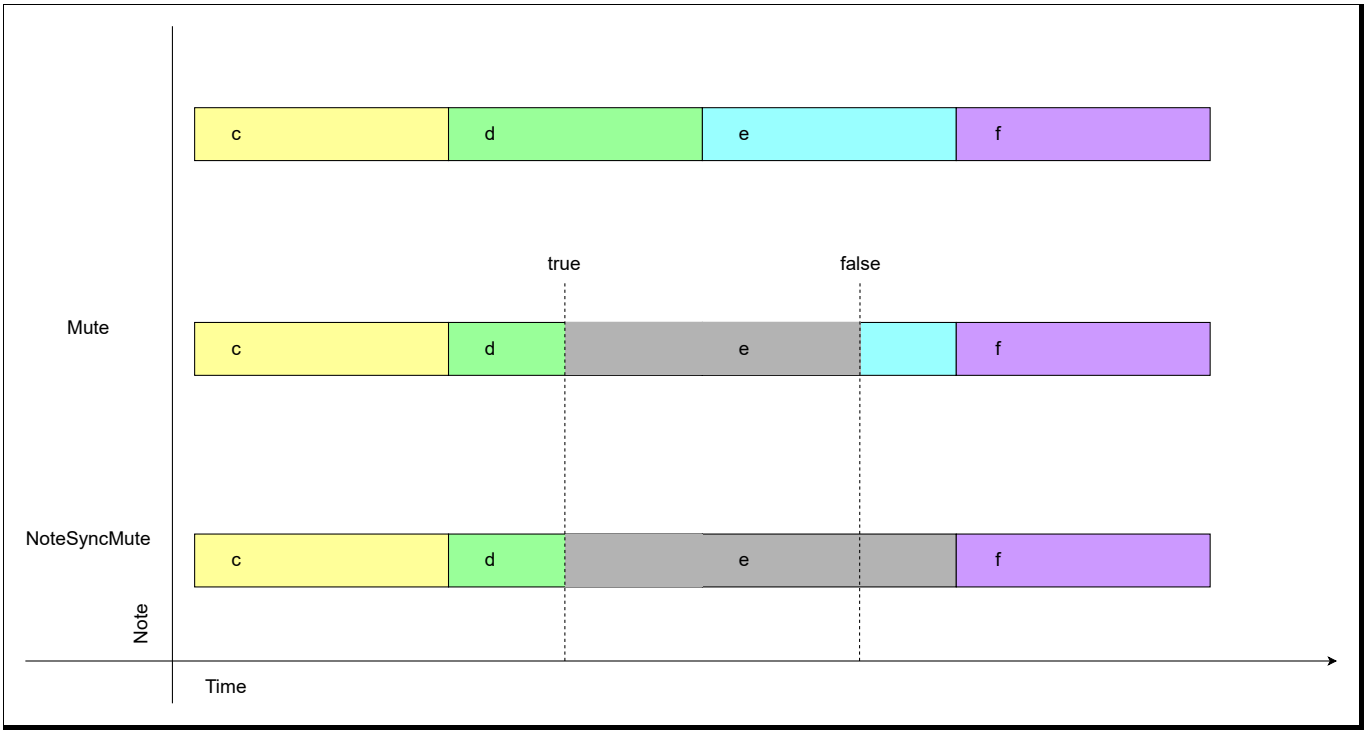
NoteSyncMute(bool isOn)

```
public void NoteSyncMute(bool isOn);
```

`v0.9.7beta`

- パラメーター：**isOn** `True`ならミュートオン

`True`で即時にAudioSourceのミュートをオンにしますが、`False`を指定すると次のノートイベントが発生するタイミングでミュートがオフになります。
これによって、昔のゲーム機のような効果音の割り込みを表現できます。



ExportSeqJson(bool _prettyPrint)

```
public string ExportSeqJson(bool _prettyPrint)
```

`v0.9.3beta`

- パラメーター：**_prettyPrint**　`True`で改行やインデントが有効（省略時`False`）
- 戻り値：**JSON形式の文字列**

デコードされたMMLのシーケンスデータをJSON化して文字列として出力します。
シーケンスデータが無い（DecodeMML()やPlay()を行ってない）場合はNullが戻ります。
JSONの中身は、tickPerNoteの値とseqListを組み合わせたものになります。

DecodeAndExportSeqJson(bool _prettyPrint)

```
public string DecodeAndExportSeqJson(bool _prettyPrint)
```

`v0.9.3beta`

- パラメーター：**_prettyPrint**　`True`で改行やインデントが有効（省略時`False`）
- 戻り値：**JSON形式の文字列**

MMLをデコードしてからシーケンスデータをJSON化して出力します。

GetSeqJson()

```
public SeqJson GetSeqJson()
```

`v0.9.3beta`

- パラメーター：なし
- 戻り値：**SeqJsonクラスオブジェクト**

デコードされたMMLのシーケンスデータをSeqJsonクラスオブジェクトとして出力します。
ExportSeqJson()でJSON化する前のデータです。
主にMML Splitterでマルチチャンネル JSONをエクスポートする際に利用します。

ImportSeqJson(string _jsonString)

```
public bool ImportSeqJson(string _jsonString)
```

`v0.9.3beta`

- パラメーター：**_jsonString**　JSON形式の文字列
- 戻り値：インポート成功で`True`

JSON形式の文字列をシーケンスデータとしてインポートします。
この際、tickPerNoteの値も読み込まれます。

SetSeqJson(SeqJson _seqJson)

```
public bool SetSeqJson(SeqJson _seqJson)
```

`v0.9.3beta`

- パラメーター：**_seqJson**　SeqJsonクラスオブジェクト
- 戻り値：インポート成功で`True`

SeqJsonクラスオブジェクトから直接tickPerNoteの値とシーケンスデータを読み込みます。
主にMML Splitterでマルチチャンネル JSONをインポートする際に利用します。

RenderSequenceTodClipData()

```
public float[] RenderSequenceTodClipData()
```

`v0.9.4beta`

- パラメーター：なし
- 戻り値：**float配列で構成されるサンプルデータ**

シーケンス全体の波形データを生成します。
sampleRateで指定されたサンプルレートで1サンプル毎に計算し、各サンプルをfloat配列にして返します。
サンプルレートが高いほど、そしてシーケンスが長いほど、レンダリングに時間がかかります。
また、シーケンスのループコマンド（MMLのループ"L"）は、レンダリング時は無効になります。

`v0.9.6beta`

生成されたクリップデータはrenderedDatasにも入ります。

ExportRenderedAudioClip(bool isAsyncRendered)

```
public AudioClip ExportRenderedAudioClip(bool isAsyncRendered)
```

`v0.9.4beta`

- パラメーター：なし
- 戻り値：**AudioClip**

シーケンス全体の波形データを生成し、AudioClipにしてエクスポートします。
AudioClipのサンプルレートはsamplRateで指定した値になります。
効果音などレスポンスを重視したい場合に、この関数でAudioClipを先に用意しておくことができます。
逆にBGMなど長いシーケンスの場合は、レンダリングに時間がかかるので注意が必要です。

`v0.9.6beta`

- パラメーター：**isAsyncRendered**　`True`でrenderdDatasを使用（省略時`False`）
- 戻り値：**AudioClip**

レンダリングを行わずにrenderedDatasを使ってAudioClipをエクスポートします。

RenderSeqToClipDataAsync(int interruptSample)

```
public bool RenderSeqToClipDataAsync(int interruptSample)
```

`v0.9.6beta`

- パラメーター：**interruptSample**　処理が中断されるサンプル数
- 戻り値：レンダリングの開始に成功した場合に`True`

シーケンス全体の波形データの非同期レンダリングを開始します。
メインスレッドのUpdate()内などでレンダリングを監視してください。 レンダリングの進捗率はasyncRenderProgressで0から1の範囲で増加します。
レンダリングが完了するとasyncRenderIsDoneが`True`になります。
生成されたクリップデータはrenderedDatasに入ります。
ExportRenderedAudioClip(true)でrenderedDatasをAudioClipに変換できます。
なお、シーケンスのループコマンド（MMLのループ"L"）は、レンダリング時は無効になります。

CalcSeqSample()

```
public int CalcSeqSample()
```

`v0.9.6beta`

- パラメーター：なし
- 戻り値：レンダリング後のサンプル数

レンダリングした場合に生成されるクリップデータのサイズを計算します。

PlayRenderedClipData(bool isLoop)

```
public bool PlayRenderedClipData(bool isLoop)
```

`v0.9.6beta`

- パラメーター：**isLoop**　`True`ならループ再生
- 戻り値：再生に成功した場合に`True`

renderedDatasを使ってAudioClipを生成し、PSG Playerに指定されているAudioSourceで再生します。

[class] MML Splitter

[MML Splitter] 変数・Public関数一覧

- 変数
  - psgPlayers
  - multiChMMLString
  - asyncMultiRenderIsDone
  - asyncMultiRenderProgress
- Public関数
  - SplitMML()
  - SplitMML(string _multiChMMLString)
  - SetAllChannelsSampleRate(int _rate)
  - SetAllChannelClipSize(int _msec)
  - PlayAllChannels()
  - PlayAllChannelsDecoded()
  - PlayAllChannelsSequence()
  - DecodeAllChannels()
  - StopAllChannels()
  - IsAnyChannelPlaying()
  - MuteChannel(int channel, bool isOn)
  - NoteSyncMuteChannel(int channel, bool isOn)
  - ExportMultiSeqJson(bool _prettyPrint)
  - DecodeAndExportMultiSeqJson(bool _prettyPrint)
  - ImportMultiSeqJson(string _jsonString)
  - ExportMixedAudioClip(int _sampleRate, bool isAsyncRendered)
  - RenderMultiSeqToClipDataAsync(int _sampleRate, int interruptSample)
  - PlayAllChannelsRenderedClipData(bool isLoop)

[MML Splitter] 変数

---

psgPlayers

```
[SerializeField] private PSGPlayer[] psgPlayers;
```

MMLを分割送信するPSG Playerコンポーネントをチャンネルの数だけ登録します。

---

multiChMMLString

```
public string multiChMMLString;
```

分割送信する元のMML文字列を登録します。

---

asyncMultiRenderIsDone

```
public bool asyncMultiRenderIsDone { get; private set; } = false;
```

`v0.9.6beta`

全てのチャンネルで非同期レンダリングが完了すると`True`になります。
RenderMultiSeqToClipDataAsync()を呼び出した後に使用します。

---

asyncMultiRenderProgress

```
public float asyncMultiRenderProgress { get; private set; } = 0f;
```

`v0.9.6beta`

全てのチャンネルの非同期レンダリングの進捗率で0から1の範囲で増加します。
RenderMultiSeqToClipDataAsync()を呼び出した後に使用します。

[MML Splitter] Public関数

SplitMML()

```
public void SplitMML();
```

- パラメーター：なし

multiChMMLStringのMML文字列を分割して、psgPlayersに登録されてるPSG Playerに送信します。
送信チャンネルの振り分けについては「MMLリファレンス」を参照してください。

SplitMML(string _multiChMMLString)

```
public void SplitMML(string _multiChMMLString);
```

- パラメーター：**_multiChMMLString**　マルチチャンネルのMML文字列

パラメーターの引数をmultiChMMLString変数に渡して、MMLをPSG Playerに分割送信します。

SetAllChannelsSampleRate(int _rate)

```
public void SetAllChannelsSampleRate(int _rate);
```

- パラメーター：**_rate**　サンプルレート

全てのPSG Playerのサンプルレートを設定します（単位Hz）。

SetAllChannelClipSize(int _msec)

```
public void SetAllChannelClipSize(int _msec);
```

- パラメーター：**_msec**　AudioClipの長さ

全てのPSG PlayerのAudioClip長さを設定します（単位ミリ秒）。

PlayAllChannels()

```
public void PlayAllChannels();
```

- パラメーター：なし

全てのPSG Playerで同時にMMLをデコードして再生します。
マルチチャンネルMMLの分割送信は行わないので、事前にSplitMML()を実行してください。

PlayAllChannelsDecoded()

```
public void PlayAllChannelsDecoded();
```

- パラメーター：なし

全てのPSG Playerで同時にデコード済みのシーケンスデータを再生します。

PlayAllChannelsSequence()

```
public void PlayAllChannelsSequence()
```

- パラメーター：なし

全てのPSG Playerで同時にシーケンスデータを再生します。
PlayAllChannelsDecoded()と同じです。

DecodeAllChannels()

```
public void DecodeAllChannels()
```

- パラメーター：なし

全てのPSG PlayerでMMLをシーケンスデータにデコードします。
マルチチャンネルMMLの分割送信は行わないので、事前にSplitMML()を実行してください。

StopAllChannels()

```
public void StopAllChannels();
```

- パラメーター：なし

全てのPSG Playerの再生を停止します。

IsAnyChannelPlaying()

```
public bool IsAnyChannelPlaying();
```

- パラメーター：なし
- 戻り値：いずれかのPSG Playerが再生中なら`True`

各PSG PlayerのAudioSourceのうち、どれか一つでも再生中なら`True`を返します。

MuteChannel(int channel, bool isOn)

```
public void MuteChannel(int channel, bool isOn);
```

- パラメーター：**channel**　対象チャンネル
  isOn　`True`ならミュートオン

指定したチャンネルをミュートします。

NoteSyncMuteChannel(int channel, bool isOn)

```
public void NoteSyncMuteChannel(int channel, bool isOn);
```

`v0.9.7beta`

- パラメーター：**channel**　対象チャンネル
  isOn　`True`ならミュートオン

指定したチャンネルでNoteSyncMuteを設定します。

ExportMultiSeqJson(bool _prettyPrint)

```
public string ExportMultiSeqJson(bool _prettyPrint)
```

`v0.9.3beta`

- パラメーター：**_prettyPrint**　`True`で改行やインデントが有効（省略時`False`）
- 戻り値：**JSON形式の文字列**

各PSG Playerのデコードされたの MMLのシーケンスデータをまとめてJSON化して文字列として出力します。
JSONの中身は、各PSG Playerで出力したSeqJsonクラスオブジェクトをリスト化したものになります。

DecodeAndExportMultiSeqJson(bool _prettyPrint)

```
public string DecodeAndExportMultiSeqJson(bool _prettyPrint)
```

`v0.9.3beta`

- パラメーター：**_prettyPrint**　`True`で改行やインデントが有効（省略時`False`）
- 戻り値：**JSON形式の文字列**

各PSG PlayerのMMLをデコードしてからシーケンスデータをJSON化して出力します。
マルチチャンネルMMLの分割送信は行わないので、事前にSplitMML()を実行してください。

ImportMultiSeqJson(string _jsonString)

```
public void ImportMultiSeqJson(string _jsonString)
```

`v0.9.3beta`

- パラメーター：**_jsonString**　JSON形式の文字列

JSON形式の文字列をマルチチャンネルのシーケンスデータとしてインポートします。

ExportMixedAudioClip(int _sampleRate, bool isAsyncRendered)

```
public AudioClip ExportMixedAudioClip(int _sampleRate, bool isAsyncRendered)
```

`v0.9.4beta`

- パラメーター：**_sampleRate**　AudioClipのサンプルレート
- 戻り値：**AudioClip**

各PSG Playerでレンダリングした波形データをミックスしてAudioClipをエクスポートします。
混合した波形データがサンプル値の範囲を超えないように、チャンネル数で割った値で合成します。
そのため、単音の最大音量はチャンネル数によって減少します。
また、全チャンネルでサンプルレートを揃える必要があるので、この関数を使用後は各PSG PlayerのsampleRateが引数で指定した値になります。
全部のPSG Playerでレンダリングした後にミックスするので、特に長いシーケンスの場合は処理に時間がかかるので注意してください。

`v0.9.6beta`

- パラメーター：**_sampleRate**　AudioClipのサンプルレート
- パラメーター：**isAsyncRendered**　`True`でレンダリング済みの波形データを使用（省略時`False`）
- 戻り値：**AudioClip**

isAsyncRenderedを`True`にすると、各チャンネルでレンダリングしたrenderedDatasをミックスしてAudioClipをエクスポートします。
この場合レンダリングはしないので、RenderMultiSeqToClipDataAsync()を呼び出した後、asyncMultiRenderIsDoneが`True`になってから使用してください。

RenderMultiSeqToClipDataAsync(int _sampleRate, int interruptSample)

```
public bool RenderMultiSeqToClipDataAsync(int _sampleRate, int interruptSample)
```

`v0.9.6beta`

- パラメーター：**_sampleRate**　AudioClipのサンプルレート
- パラメーター：**interruptSample**　処理が中断されるサンプル数
- 戻り値：いずれかのチャンネルでレンダリングが開始したら`True`

全チャンネルで非同期レンダリングを開始します。
メインスレッドのUpdate()内などでレンダリングを監視してください。
レンダリングの進捗率はasyncMultiRenderProgressで0から1の範囲で増加します。

レンダリングが完了するとasyncMultiRenderIsDoneが`True`になります。
生成されたクリップデータは各PSG Playerのrenderedに入ります。
ExportMixedAudioClip()でrenderedをミックスしてAudioClipに変換できます。
なお、シーケンスのループコマンド（MMLのループ"L"）は、レンダリング時は無効になります。

PlayAllChannelsRenderedClipData(bool isLoop)

```
public void PlayAllChannelsRenderedClipData(bool isLoop)
```

`v0.9.6beta`

- パラメーター：**isLoop**　`True`ならループ再生
- 戻り値：なし

各チャンネルでrenderedを使ってAudioClipを生成し、それぞれのPSG Playerに指定されているAudioSourceで再生します。
ミックスされずに鳴らすので、各チャネルの音量変更やミュートをすることができます。